M3 Showcase — Project Documentation

Nov 13, 2026 · @Koemheang Am

Overview

M3 Showcase is a single-module Android banking demo built with Jetpack Compose and Material 3 Expressive. It shows a production-style Clean Architecture app: PIN login, accounts, transfers, receipts, QR receive, festival themes, offline handling and an auto-lock session timer. All banking data comes from a fake in-process backend, so the app runs with no server.

What a user can do

- Sign in with a 4-digit PIN (demo PIN `1234`), with haptic feedback on every key.
- See a Home dashboard: account carousel, quick actions, recent transactions.
- Browse accounts and filter transaction history per account.
- Transfer money between accounts and get a receipt.
- Show a QR code to receive money.
- Change light/dark mode and a Cambodian festival skin (Water Festival, Khmer New Year, Pchum Ben, Independence Day, Winter, Spring) or let it follow the calendar.
- Hide balances, set the auto-lock time (5 s, 5, 10 or 15 min) and log out.
- Explore a Material 3 Expressive component showcase.

Tech stack

Area	Choice	Version
Language	Kotlin (JVM target 11)	2.2.10
UI	Jetpack Compose (BOM)	2026.02.01
Design	Material 3 Expressive, graphics-shapes	1.5.0-beta01, 1.1.0
Navigation	Navigation 3 (we own the back stack)	1.2.0
DI	Koin (BOM)	4.2.2
Async	Kotlin Coroutines + Flow	1.10.2
Lifecycle	lifecycle-runtime / viewmodel-navigation3	2.11.0
Storage	DataStore Preferences	1.2.1
Serialization	kotlinx.serialization (nav keys)	1.9.0

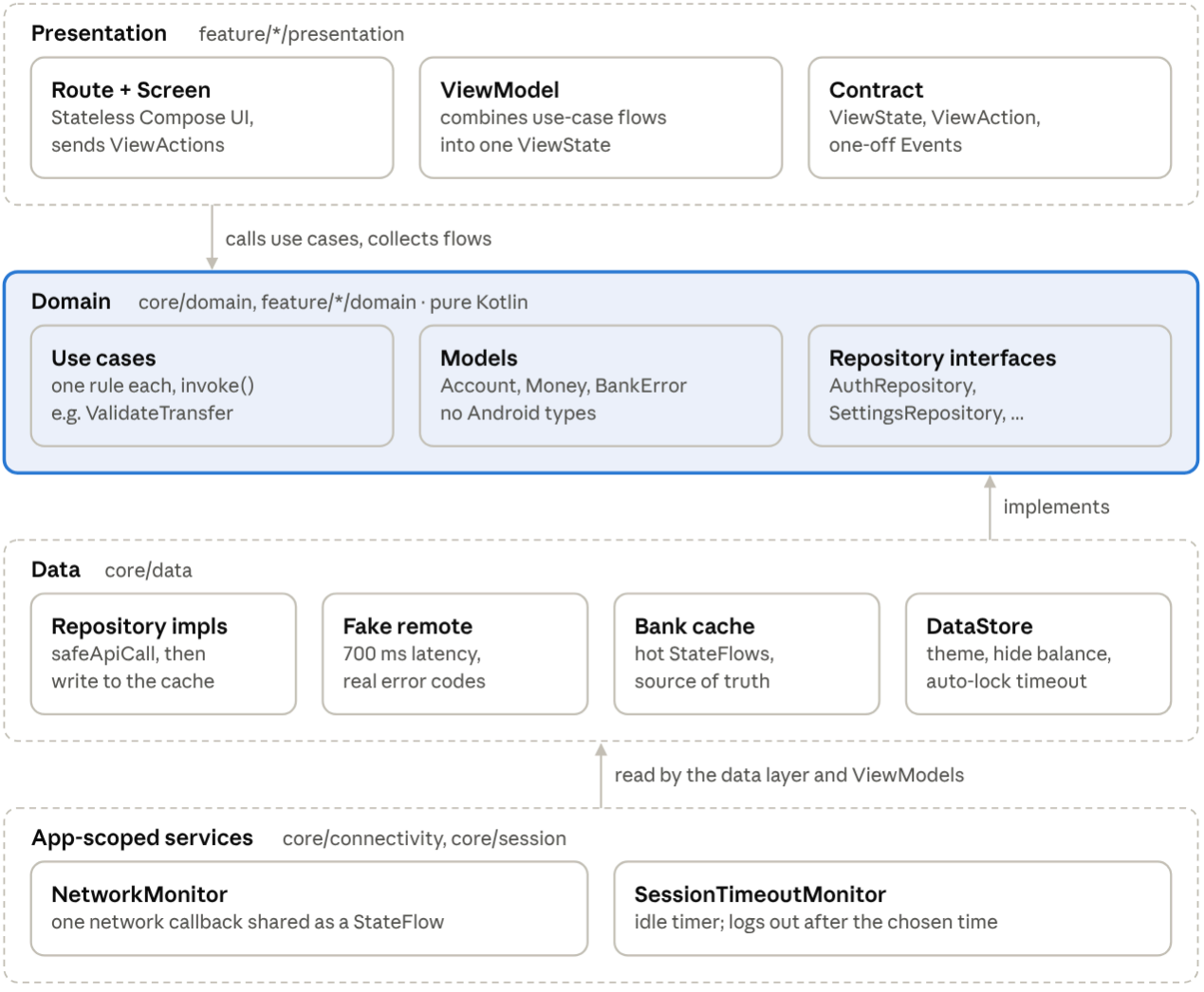
Area	Choice	Version
QR codes	ZXing core	3.5.3
Build	AGP, compileSdk / targetSdk 37, minSdk 24	9.4.1
Tests	JUnit 4, kotlinx-coroutines-test	4.13.2

Package root: `com.example.m3showcase`. Application id: `com.example.m3showcase`.

Architecture

The app uses Clean Architecture in three layers. Dependencies point inward to the domain: presentation calls domain use cases, and data implements domain interfaces. The domain knows nothing about Compose, Android or Koin.

Screens call use cases; the data layer implements the domain interfaces



architecture · 3 layers + app-scoped services

Screens only call use cases, never repositories or data sources. So the fake remote could be swapped for Retrofit, or the cache for Room, without touching a screen.

Package structure

```
com.example.m3showcase
├── ShowcaseApp.kt            Application: starts Koin
├── MainActivity.kt           single Activity, theme, auto-lock hooks
├── main/                     BankApp (NavDisplay + bottom bar),
MainViewModel
├── core/
|   ├── common/               DateProvider, DispatcherProvider,
MonotonicClock, MoneyFormatter
|   ├── connectivity/         ConnectivityObserver, NetworkMonitor
|   ├── session/              SessionTimeoutMonitor
|   ├── data/                 remote (fake API, DTOs), local (cache,
DataStore), mappers, repositories
|   ├── domain/               models, repository interfaces, shared use cases
|   ├── designsystem/         theme, colors, festival art, AppBackground,
components
|   ├── navigation/           Destination, TopLevelDestination, AppNavigator
|   ├── ui/                   labels and formatters (toUserMessage,
displayName)
|   └── di/                   coreModule
└── feature/
    ├── auth/  home/  accounts/  transfer/  receipt/  receive/
    ├── settings/  showcase/  report/
    └── <each>/ di/ · domain/ · presentation/
```

Dependency rules

- `feature/*` may use `core/*`, but features never import each other. Shared logic moves to `core`.
- Domain code imports no Android classes, so it runs in plain JVM unit tests.
- Only the data layer knows about DTOs; mappers convert at the border.
- Only `AppNavigator` changes the back stack.

App startup and dependency injection

The app starts in `ShowcaseApp`, which starts Koin with one core module, one main module and one module per feature. Everything after that is resolved by Koin: no class creates its own dependencies.

Startup sequence

1. `ShowcaseApp.onCreate()` calls `startKoin { androidContext(...); modules(coreModule, mainModule, authModule, homeModule, accountsModule, transferModule, receiptModule, receiveModule, settingsModule, showcaseModule) }`.
2. `MainActivity.onCreate()` enables edge-to-edge and gets an Activity-scoped `MainViewModel` with `by viewModel()`.
3. `MainViewModel.uiState` starts as `Loading`. While it is `Loading`, `setContent` draws nothing, so the theme never flashes the wrong colors.
4. Once settings and session arrive, the state becomes `Ready(themeMode, eventTheme, isLoggedIn)`. The Activity applies system-bar styles and draws `AppTheme { BankApp(...) }`.
5. `MainActivity.onUserInteraction()` and `onStart()` forward to `MainViewModel` to drive the auto-lock timer (see the Session timeout section).

Koin modules

Module	File	Provides
`coreModule`	`core/di/CoreModule.kt`	Dispatchers, date provider, monotonic clock, app-wide `CoroutineScope` (`AppScope`), connectivity observer + `NetworkMonitor`, `SessionTimeoutMonitor`, data sources, all repositories, shared use cases
`mainModule`	`main/MainModule.kt`	`MainViewModel`
`authModule`	`feature/auth/di`	`LoginWithPinUseCase`, `LoginViewModel`
`homeModule`	`feature/home/di`	`ObserveHomeDashboardUseCase`, `HomeViewModel`
`accountsModule`	`feature/accounts/di`	history use case, `AccountsViewModel`, `TransactionHistoryViewModel`
`transferModule`	`feature/transfer/di`	`ValidateTransferUseCase`, `MakeTransferUseCase`, `TransferViewModel`
`receiptModule`	`feature/receipt/di`	`ReceiptViewModel`
`receiveModule`	`feature/receive/di`	`ObserveReceiveQrUseCase`, `ReceiveViewModel`
`settingsModule`	`feature/settings/di`	`SettingsViewModel`, `ThemeViewModel`

Module	File	Provides
`showcaseModule`	`feature/showcase/di`	showcase repository, use case, ViewModel

Scoping rules used in the project

- `single` for anything that holds state shared by the whole app: repositories, the in-memory cache, `NetworkMonitor`, `SessionTimeoutMonitor`.
- `factory` for use cases. They hold no state, so a fresh instance each time is cheap and safe.
- `viewModelOf` for ViewModels. Each Navigation 3 entry gets its own `ViewModelStore`, so a screen's ViewModel is cleared when that screen is popped, not when the Activity dies.
- `AppScope` is a named `CoroutineScope(SupervisorJob() + Dispatchers.Default)` that lives as long as the process. Hot app-wide flows run in it.

MainViewModel combines three inputs into app-level state:

- settings (theme mode + festival theme) and session (logged in or not) → `uiState`;
- network status → `connectivityBanner` (Offline, BackOnline for 2 s, Hidden);
- `onUserInteraction()` and `onAppForegrounded()` → the session-timeout use cases.

Navigation

The app uses Navigation 3, where the back stack is a plain observable list the app owns. Navigating means editing that list, and only `AppNavigator` edits it.

Destinations (`core/navigation/Destination.kt`) are `@Serializable` `NavKey` objects, so the back stack survives rotation and process death and arguments are type-safe properties.

Destination	Kind	Arguments
`Login`	Auth	none
`Home`, `Accounts`, `Receive`, `More`	Bottom-bar tabs	none
`TransactionHistory`	Detail	`accountId: String?` (null = all accounts)
`Transfer`	Detail	`fromAccountId: String?`

Destination	Kind	Arguments
`Receipt`	Detail	`transactionId: String`, `isNewTransfer: Boolean`
`Theme`, `Showcase`	Detail	none

AppNavigator (`core/navigation/AppNavigator.kt`) has five operations:

- `navigate(dest)` pushes a screen, ignoring a duplicate of the top entry.
- `goBack()` pops, but never the last entry.
- `selectTopLevel(dest)` switches tabs. Home always stays at the root, so the stack is `[Home]` or `[Home, Tab]` and Back from any tab goes Home.
- `popToHome()` resets to `[Home]`.
- `replaceAll(list)` swaps the whole stack, used for login/logout and after a transfer.

NavDisplay setup (`main/BankApp.kt`)

- `rememberNavBackStack(Destination.Login)` creates the saveable stack.
- Two entry decorators: `rememberSaveableStateHolderNavEntryDecorator()` keeps each screen's saved UI state, and `rememberViewModelStoreNavEntryDecorator()` gives each entry its own ViewModels.
- Tabs use a `TabTransition` crossfade. Other screens use the Navigation 3 default fade.
- The bottom `ShortNavigationBar` slides in only when the top entry is one of the four tabs.
- Screens that need arguments get them from the key; Koin passes them to the ViewModel with `parametersOf(...)` (Transfer, Receipt, TransactionHistory).

Session-driven auth navigation. Login does not navigate on success. Instead `BankApp` watches `isLoggedIn`:

- logged in while on Login → `replaceAll([Home])`;
- logged out anywhere else → `replaceAll([Login])`.

This single rule handles PIN login, the Log out button, the auto-lock timeout and process death with a restored back stack. A restored stack can never show bank data without a session.

Transfer → Receipt. After a successful transfer the stack becomes `[Home, Receipt(isNewTransfer = true)]`, so Back cannot return to the form and resubmit it.

Data layer

The data layer is offline-first: screens observe a local cache, and network calls only refresh that cache. A fake remote stands in for a real API, with the same failure modes, so swapping in Retrofit changes nothing above this layer.

Parts (all under `core/data/`)

Part	File	Role
Fake remote	`remote/FakeBankRemoteDataSource.kt`	In-memory "server": user, 3 accounts, seeded transactions, transfer logic
DTOs	`remote/dto/BankDtos.kt`	Wire shapes: `UserDto`, `AccountDto`, `TransactionDto`, `TransferRequestDto`, `TransferResponseDto`
Error funnel	`remote/SafeApiCall.kt`	Turns every exception into a `Result` with a `BankError`
Mappers	`mapper/BankMappers.kt`	DTO ↔ domain (`toDomain()`, `toDto()`), string codes → enums
Bank cache	`local/BankLocalDataSource.kt`	Hot `StateFlow`s of accounts and transactions; the single source of truth
Settings store	`local/SettingsLocalDataSource.kt`	DataStore Preferences file `user_settings`
Repositories	`repository/RepositoryImpls.kt`	Implement the domain interfaces by combining the above
Festival calendar	`repository/StaticFestivalCalendarRepository.kt`	Hard-coded festival date windows per year

How the fake remote behaves like a network. Every call goes through `request(latencyMs = 700)`: it throws `IOException` if `NetworkMonitor` says offline, waits for the latency, checks connectivity again, then does the work under a `Mutex`. Transfers take 1,200 ms and enforce real rules: unknown account (`ACCOUNT_NOT_FOUND`), same account (`SAME_ACCOUNT`), different currency (`CURRENCY_MISMATCH`) and not enough money (`INSUFFICIENT_FUNDS`). A transfer between the user's own accounts also credits the destination.

Error handling. `safeApiCall { ... }` is the only place exceptions become domain errors:

Thrown	Becomes
`CancellationException`	rethrown, so coroutine cancellation keeps working
`SocketTimeoutException`	`BankError.Timeout`
`IOException`	`BankError.NoInternet`
`ApiException` with a code	`InvalidPin`, `InsufficientFunds`, `AccountNotFound` or `Validation(field, reason)`
anything else	`BankError.Unknown(message)`

Repository styles. Two styles on purpose:

- `observe...()` returns a `Flow` from the cache. The UI subscribes once and re-renders whenever the cache changes.
- `suspend` one-shot calls (`login`, `refreshAccounts`, `transfer`) hit the remote and return `Result`. On success they write into the cache, which updates every observing screen. After a transfer, Home's balances update by themselves.

Cache details. `null` means never loaded, which differs from loaded but empty; repositories filter out `null`. `upsertAccounts` replaces changed accounts by id, `addTransactions` prepends and re-sorts newest first, and `clear()` wipes everything on logout. To persist data across restarts, replace it with Room DAOs, which also return Flows.

Settings storage. DataStore keys: `theme_mode`, `balance_hidden`, `event_theme`, `session_timeout`. Enums are saved by name, and an unknown name (for example a theme removed in a later version) falls back to the default. IO errors emit empty preferences instead of crashing.

Money. Amounts are `Money(minor: Long, currency)`, in cents for USD and riel for KHR, so balances never use floating point. Adding two different currencies throws.

Domain layer

The domain layer is pure Kotlin: models, repository interfaces and use cases, with no Android or UI types. Business rules live here, so ViewModels stay thin and the rules are unit-testable.

Models (`core/domain/model/`)

Model	Meaning
`UserProfile`	id, full name, phone number
`Account`	id, name, 9-digit number, `AccountType` (Savings, Current, FixedDeposit), `Money` balance, `isDefault`
`Transaction`	signed `Money` amount (negative = money out), category, timestamp, optional note; `isCredit`
`TransferRequest`	from account id, to account number, amount, note
`UserSettings`	theme mode, balance hidden, festival theme, session timeout
`ThemeMode`	System, Light, Dark
`EventTheme`	Auto, Default, WaterFestival, KhmerNewYear, PchumBen, IndependenceDay, Winter, Spring
`FestivalWindow`	holiday start/end plus a `showFrom` date a few days earlier
`SessionTimeout`	FiveSeconds, FiveMinutes (default), TenMinutes, FifteenMinutes
`Money` / `CurrencyCode`	minor units; USD (2 decimals) and KHR (0 decimals)
`BankError`	every failure the UI can see (see Data layer)

Repository contracts (`core/domain/repository/Repositories.kt`): `AuthRepository` (hot `session: StateFlow<UserProfile?>` , `login` , `logout`), `AccountRepository` , `TransactionRepository` , `TransferRepository` , `SettingsRepository` (cold `settings: Flow<UserSettings>` + setters) and `FestivalCalendarRepository` . The showcase feature has its own `ShowcaseRepository` .

Use cases. Each one does one thing through a single `operator fun invoke` .

Use case	Where	What it does
`ObserveAccountsUseCase`	core	Accounts sorted default-first, then by name

Use case	Where	What it does
`RefreshBankDataUseCase`	core	Refreshes accounts and transactions in parallel; fails if either fails
`GetTransactionUseCase`	core	One transaction from the cache, or `NotFound`
`ObserveSettingsUseCase` / `SetThemeModeUseCase` / `SetBalanceHiddenUseCase` / `SetEventThemeUseCase` / `SetSessionTimeoutUseCase`	core	Read and write settings
`ObserveSessionUseCase` / `LogoutUseCase`	core	Session state and logout
`ObserveNetworkStatusUseCase`	core	App-wide connectivity `StateFlow`
`GetActiveFestivalUseCase`	core	Today's festival window, if any (Cambodia time zone)
`ResolveEventThemeUseCase`	core	Auto → today's festival or Default; an explicit pick always wins
`GetFestivalScheduleUseCase`	core	Next dates of each festival for the theme picker
`RecordUserActivityUseCase` / `CheckSessionExpiryUseCase`	core	Drive the auto-lock timer
`LoginWithPinUseCase`	auth	Checks 4 digits, then logs in
`ObserveHomeDashboardUseCase`	home	Combines accounts, 5 latest transactions, user and hide-balance flag
`ObserveTransactionHistoryUseCase`	accounts	One account's (or all) transactions with an All / MoneyIn / MoneyOut filter
`ValidateTransferUseCase`	transfer	All form rules (below)
`MakeTransferUseCase`	transfer	Sends the validated request
`ObserveReceiveQrUseCase`	receive	One QR payload per account
`GetShowcaseContentUseCase`	showcase	Static showcase content

Transfer validation rules (`ValidateTransferUseCase`), checked in this order:

1. A source account must be chosen.
2. The recipient must be exactly 9 digits and not the source account.
3. The amount must parse, be greater than zero and have at most the currency's decimals (2 for USD, 0 for KHR).
4. The amount must not exceed the per-transaction limit: USD 10,000 or KHR 40,000,000.
5. The amount must not exceed the balance.
6. The note is trimmed, cut to 100 characters, and dropped if blank.

Each failure is a `BankError.Validation(field, reason)` , so the screen can show the message under the right field.

Presentation pattern

Every screen follows the same unidirectional pattern: the screen sends a **ViewAction**, the ViewModel calls use cases and emits a new **ViewState**, and one-off effects travel as **Events**. Screens contain no business logic.

The four pieces of every feature

Piece	File	Rule
ViewState	`XxxContract.kt`	`@Immutable` . A sealed interface (Loading / Success / Empty / Error) for data screens, or one data class for forms such as Login
ViewAction	`XxxContract.kt`	A sealed interface of user intents, named `OnSomethingClicked` / `OnSomethingChanged`
Event	`XxxContract.kt`	One-off effects: navigation, snackbars. Sent through a `Channel(BUFFERED).receiveAsFlow()` , so each event reaches exactly one collector and is not lost while the screen is in the background
ViewModel	`XxxViewModel.kt`	Exposes `viewState: StateFlow` and `events: Flow` , plus one public `fun onAction(action)`

Route vs Screen. Each feature has two composables:

- `XxxRoute` gets the ViewModel from Koin, collects state with `collectAsStateWithLifecycle()` , collects events in a `LaunchedEffect` and turns them into navigation callbacks.

- `XxxScreen(state, onAction)` is stateless. It can be previewed and tested with any state.

Building state from streams. ViewModels `combine` cold data streams (cache, DataStore), hot app-wide streams (network, session) and a local `MutableStateFlow` for UI-only input, then call `stateIn(viewModelScope, SharingStarted.WhileSubscribed(5_000), Loading)`. The 5-second window keeps the upstream alive across a rotation but stops it soon after the screen leaves.

Walkthrough: Transfer

1. State. `TransferViewState` is `Loading`, `NoAccounts` or `Form(accounts, selectedAccount, recipient, amount, note, recipientError, amountError, isOnline, isSubmitting, pendingConfirmation)`. `Form.canContinue` is true only when online, not submitting and both fields are filled.
2. Inputs. The ViewModel keeps what the user typed in a private `FormInput` `MutableStateFlow`. It combines that with `observeAccounts()` and `observeNetworkStatus()` into one `Form` state. The selected account falls back to the default account, then to the first.
3. Typing. `OnRecipientChanged` keeps digits only, at most 9. `OnAmountChanged` keeps digits and one decimal point, at most 12 characters. Editing a field clears its error.
4. Continue. `OnContinueClicked` runs `ValidateTransferUseCase`. Success sets `pendingConfirmation`, which opens the confirmation bottom sheet. A validation error lands under the matching field.
5. Confirm. `OnConfirmClicked` ignores double taps (`isSubmitting` guard), calls `MakeTransferUseCase`, then sends `TransferEvent.OpenReceipt(txId)`. `TransferRoute` turns that into `replaceAll([Home, Receipt])`.
6. Errors. Insufficient funds shows under the amount. Other errors (no internet, timeout) become a `ShowMessage` snackbar event.

Error messages. `BankError.toUserMessage()` (in `core/ui/UiFormatters.kt`) is the single place that turns domain errors into user-facing text.

Feature tour

Each feature lives in `feature/<name>/` with its own `di`, optional `domain`, and `presentation` packages. The table lists every screen, its states and what it does.

Screen	Package	States	What it does
Login (lock screen)	`auth`	one form data class: `pin`, `isLoading`, `error`, `failureCount`	4-digit PIN pad. Submits automatically at the 4th digit. A wrong PIN clears the dots, shakes them and gives a reject vibration; each key gives a key-tap vibration. "Forgot PIN" shows branch contact info. Festival hero art draws behind it.
Home	`home`	Loading, Error(offline?), Success(dashboard, isRefreshing)	Navy hero header with greeting and account carousel, an 8-button quick-action grid, 5 latest transactions, pull to refresh, hide-balance toggle. Refreshes on start and again automatically when the network returns after a failed refresh. Payments, Cards, Deposits, Loans and Locator show "coming soon".
Accounts	`accounts/list`	Loading, Empty, Error, Success	Card per account with balance and a Transfer button; tap a card for its history. Large flexible top bar that collapses on scroll.
Transaction history	`accounts/history`	Loading, Error, Success, each carrying the filter	All, Money in or Money out filter; one account or all accounts; tap a row for its receipt.

Screen	Package	States	What it does
Transfer	`transfer`	Loading, NoAccounts, Form	Source account picker, 9-digit recipient, amount with quick-amount chips, note, confirmation bottom sheet; Continue is disabled offline.
Receipt	`receipt`	Loading, Error, Success(transaction, isNewTransfer)	Success check mark for a new transfer, full details, Share via the Android share sheet. Close goes Home for a new transfer, otherwise Back.
Receive QR	`receive`	Loading, Empty, Error, Success(codes, selectedIndex)	One QR code per account, drawn with ZXing, payload `novabank://pay?acc=...`; account switcher and Share.
Trigger Report	`report`	static sample rows	A report table whose "No." and "Trigger Type" columns stay fixed while Total and the day columns scroll sideways.
More (Settings)	`settings`	Loading, Success	Profile card, Theme, Auto-lock picker, Hide balances, link to the showcase, Log out with confirmation.
Theme	`settings/theme`	Loading, Success	Light / Dark / System, festival skins with live preview and their next dates, Auto-follow-calendar switch.

Screen	Package	States	What it does
M3 Expressive showcase	`showcase`	one state class + effects	Three tabs (Tactics & Hero, Components, Expressive Chat) demonstrating color, shape, size, motion and containment, plus a chat demo with emoji and photo pickers.

Note on the Receive tab. In `main/BankApp.kt` the bottom-bar Receive tab currently opens `TriggerReportRoute()`, not `ReceiveRoute()`. The Home "Receive QR" button goes to the same tab, so the QR screen is built but not reachable right now. To restore it, change that entry back to `ReceiveRoute()` or give the report its own destination.

Design system

The look is a navy-and-cyan banking brand on Material 3 Expressive, with a festival skin layered on top and one shared animated background. Dynamic (wallpaper) color is off on purpose, so the brand colors stay fixed.

Theme entry point. `AppTheme(darkTheme, eventTheme)` in `core/designsystem/Theme.kt` wraps `MaterialExpressiveTheme` with:

- a color scheme picked by `colorSchemeFor(event, dark)`;
- `MotionScheme.expressive()`, the spring-based motion;
- rounded shapes: 8, 12, 20, 28 and 36 dp from extra-small to extra-large;
- `BankTheme.colors`, extra brand colors Material has no slot for: `heroBrush` (navy gradient), `onHero`, `onHeroVariant`, `heroAccent` (cyan), `credit` (green) and `debit`;
- `BankTheme.eventTheme`, the festival currently drawn (never Auto).

Brand palette (`Color.kt`): NavyDeep `#041A2E`, Navy `#0A2E4D`, NavyLight `#12426B`, Cyan `#00A9CC`, CyanLight `#6FD8F0`, Coral `#E5484D`, Mint `#2FBF71`. Light primary is `#005A8C` on a `#F8F9FB` background.

Festival skins (`FestivalPalettes.kt`, `FestivalHeroArt.kt`). Each festival repaints only the accent roles (primary, secondary and their containers) and the hero gradient. Surfaces stay neutral, so text contrast never changes. Each also has animated hero art drawn with Canvas.

Skin	Inspiration	Hero art
Water Festival	night river, gold fireworks	waves and fireworks
Khmer New Year	festive red and marigold	spinning flowers
Pchum Ben	saffron robes, candlelight	lotus flowers
Independence Day	national flag blue and red	red accents
Winter	frost blue	falling snowflakes
Spring	blush pink	blossoms

With **Auto**, `ResolveEventThemeUseCase` shows a festival from its `showFrom` date (a few days early) to the end of the holiday, using Cambodia's time zone, and Default otherwise. The dates come from `StaticFestivalCalendarRepository`, because lunar holidays move every year.

App background (`AppBackground.kt`). One backdrop sits behind every screen: the theme background fading into `surfaceContainerLow`, plus three large soft glows in primary, tertiary and secondary. Each glow orbits slowly on a 30-second loop.

- Glow strength is 0.18 alpha in light mode and 0.30 in dark mode; change `glowAlpha` to tune it.
- The animation is read only inside `drawBehind`, so it redraws without recomposing any screen. It pauses in the background and respects the system "Remove animations" setting.
- It provides `onBackground` as the default content color. Without it, transparent Scaffolds fall back to black text.
- Screens use `containerColor = Color.Transparent`. Top bars use `bankTopAppBarColors()`: transparent at rest, a 94%-opaque `surfaceContainer` once content scrolls under. The bottom bar is also 94% opaque.
- Screens crossfade over a still background when you navigate.

Shared components (`core/designsystem/component/`)

Component	Use
`LoadingState`, `ErrorState`, `EmptyState`	Standard full-screen states; `ErrorState` has Retry and an offline variant
`ConnectivityBanner`	Offline / Back online strip at the top of the app

Component	Use
`TransactionItem`	One transaction row with category icon and colored amount
`QrCode`	Renders a string as a QR bitmap with ZXing
`MorphShape`	A shape that morphs between two Material shapes (graphics-shapes)
`StatusBarIcons(light)`	Switches status-bar icon color per screen (light icons over the navy hero)

Session timeout (auto-lock)

After the chosen idle time (5 s, 5, 10 or 15 min; default 5 min) the app logs the user out, and the session-driven navigation rule sends them to the PIN screen. Two paths make this reliable: a foreground timer, and a real-time check when the app returns from the background.

Pieces

Piece	File	Role
`SessionTimeout` enum	`core/domain/model/SessionTimeout.kt`	The 4 options and their `Duration`
`MonotonicClock`	`core/common/MonotonicClock.kt`	`SystemClock.elapsedRealtime()`: keeps counting while the device sleeps and ignores wall-clock changes; replaceable in tests
`SessionTimeoutMonitor`	`core/session/SessionTimeoutMonitor.kt`	App-scoped singleton holding the timer
Use cases	`core/domain/usecase/SessionTimeoutUseCases.kt`	`SetSessionTimeoutUseCase`, `RecordUserActivityUseCase`, `CheckSessionExpiryUseCase`
Hooks	`MainActivity` + `MainViewModel`	`onUserInteraction()` → record activity; `onStart()` → check expiry
Setting UI	More → Privacy & security → Auto-lock	Radio-button dialog; saved in DataStore as `session_timeout`

Path 1: foreground timer. The monitor combines three flows, `isLoggedIn`, `timeout` (from settings) and `lastActivity`, and runs `collectLatest`:

```
combine(isLoggedIn, timeout, lastActivity) { ... }
    .collectLatest { (loggedIn, timeout, last) ->
        if (!loggedIn) return@collectLatest
        delay(timeout - (clock.now() - last))
        authRepository.logout()
    }
```

Every new emission cancels the running `delay` and starts a new one. So a tap, a settings change or a login restarts the countdown, and a logout stops it. A fresh login also resets `lastActivity`, so each session starts with a full idle window.

Path 2: back from background. Coroutine `delay` uses a clock that stops in deep sleep, so the timer can fire late on a dozing phone. On every `Activity.onStart()`, `checkExpired()` compares `elapsedRealtime` against the last activity and logs out at once if the window has passed.

Why `onUserInteraction()`. Android calls it on touch-down and key-down only, not on every move event, so recording activity costs one timestamp write per tap.

Known limits

- Typing on the on-screen keyboard does not reset the timer, because IME input does not reach the Activity as key events. Taps inside the app do.
- Timeout uses the normal logout, which also clears the cached account data. A softer "locked, keep data" mode would need a separate locked state.
- Returning after a timeout can show the previous screen for one frame before Login. Hiding app content in Recents (`FLAG_SECURE`) would prevent that.
- The 5-second option is labelled "testing" and is meant for demos.

Connectivity and the offline banner

The app registers exactly one system network callback and shares its result as a hot `StateFlow`. Screens, the fake backend and the banner all read the same value.

1. **Cold source.** `AndroidConnectivityObserver.observe()` wraps `ConnectivityManager.registerDefaultNetworkCallback` in a `callbackFlow`. It emits the current status at once, unregisters in `awaitClose` so it cannot leak, and applies `distinctUntilChanged().conflate()`. A network counts as online only when it has both `INTERNET` and `VALIDATED` capabilities, so a captive Wi-Fi portal counts as offline.
2. **Hot share.** `NetworkMonitor` turns it into `status: StateFlow<NetworkStatus>` with `SharingStarted.Eagerly` in `AppScope`, and offers `isOnline` for synchronous checks. It is eager because the data layer reads `isOnline` even when no screen is

collecting.

3. **Consumers.**
   - `FakeBankRemoteDataSource` throws `IOException` when offline, which becomes `BankError.NoInternet`.
   - `TransferViewModel` disables Continue while offline.
   - `HomeViewModel` re-runs a failed refresh automatically when the network returns.
   - `MainViewModel` drives the banner.
4. **Banner logic** (`MainViewModel.connectivityBanner`). `runningFold` keeps the previous status next to the current one, and `transformLatest` maps the pair to a state:
   - now offline → `Offline`, shown until back online;
   - offline → online → `BackOnline` for 2 s, then `Hidden`;
   - online at start → `Hidden`.

   If the network drops again during the 2 seconds, `transformLatest` cancels the timer. While the banner shows, `BankApp` consumes the status-bar inset so screens below don't add it twice, and switches the status-bar icons to light.

Testing

The project has 23 JVM unit tests across 5 test classes (22 real tests plus the template `ExampleUnitTest`), and all pass. They focus on the domain rules and ViewModel logic, which is where the architecture puts the decisions. Run them with `./gradlew :app:testDebugUnitTest`.

Test class	Tests	What it proves
`core/EventThemeUseCasesTest`	7	Auto picks the festival during the holiday, switches on a few days early, turns off the day after, handles Independence Day in years without lunar dates, lets an explicit pick win, handles Pchum Ben's lead crossing into the previous month, and rolls the schedule over to next year

Test class	Tests	What it proves
`core/session/SessionTimeoutMonitorTest`	5	Logs out after the idle timeout, activity restarts the countdown, a settings change applies the new timeout, and returning from background after or before the timeout locks or keeps the session
`feature/transfer/ValidateTransferUseCaseTest`	7	USD converts to cents; KHR rejects decimals; above-balance, zero, short account number, same account and missing source account each fail on the right field
`feature/auth/LoginViewModelTest`	3	Correct PIN starts a session; a wrong PIN clears input, shows an error and bumps the shake counter; delete removes the last digit

How the tests are made testable

- Time is injected: `DateProvider` fixes "today" for the festival tests, and `MonotonicClock` is driven by the coroutine test scheduler's virtual time for the session tests. A 15-minute timeout is tested in milliseconds.
- Dispatchers are injected through `DispatcherProvider`; ViewModel tests set `Dispatchers.Main` to an `UnconfinedTestDispatcher`.
- Repositories are interfaces, so tests use small hand-written fakes (`FakeAuthRepository`, `FakeSettingsRepository`) instead of mocking libraries.
- Long-running app-scope coroutines run in `runTest`'s `backgroundScope`, which is cancelled when the test ends.

Gaps worth filling next: Home, Accounts and Transfer ViewModel tests; `safeApiCall` error mapping; the connectivity banner state machine; and Compose UI tests (only the template `ExampleInstrumentedTest` exists under `androidTest`).

Build, run and extend

Open the project root in Android Studio, or use Gradle from the terminal. The app needs no server and no API keys. The only permission is `ACCESS_NETWORK_STATE`.

Task	Command
Compile	`./gradlew :app:compileDebugKotlin`
Install on a device or emulator	`./gradlew :app:installDebug`
Unit tests	`./gradlew :app:testDebugUnitTest`
Release build (R8 enabled)	`./gradlew :app:assembleRelease`

Trying it out

- Sign in with the demo PIN **1234**. Any other 4 digits shows the wrong-PIN shake.
- Turn on airplane mode to see the offline banner, the disabled Transfer button and the no-internet errors. Turn it off to see "Back online" and Home refresh on its own.
- Set More → Auto-lock → 5 seconds (testing) and leave the phone idle to see the lock.
- Use More → Theme to try each festival skin in light and dark.

Checklist: adding a new feature

1. Add a `@Serializable` key to `Destination`, and to `TopLevelDestination` if it is a tab.
2. Domain: add models and a repository interface if new data is needed, then one use case per action under `feature/<name>/domain`.
3. Data: implement the repository in `core/data/repository` (or the feature's `data` package), wrapping remote calls in `safeApiCall` and writing results into the local cache.
4. Contract: write `XxxViewState` (Loading / Success / Empty / Error), `XxxViewAction` and `XxxEvent` in `XxxContract.kt`.
5. ViewModel: `combine` the use-case flows into `viewState` with `stateIn(WhileSubscribed(5_000))`, expose `events` from a `Channel`, and handle every action in `onAction`.
6. UI: write a stateless `XxxScreen(state, onAction)` and an `XxxRoute` that collects state and events. Give the `Scaffold` `containerColor = Color.Transparent` and the top bar `bankTopAppBarColors()` so the app background shows.
7. DI: create `feature/<name>/di/XxxModule.kt` (`factoryOf` for use cases, `viewModelOf` for the ViewModel, `parametersOf` for nav arguments) and register it in `ShowcaseApp`.
8. Navigation: add an `entry<Destination.Xxx> { XxxRoute(...) }` in `BankApp` and navigate through `AppNavigator`.
9. Tests: unit-test the use cases and the ViewModel with fakes and injected time.